AUTODOCS SISTEMA DE GESTIÓN Y APIS

Documentación Técnica

Generado: 16/10/2026 03:02 UTC

AutoDocs AI

Tabla de Contenido

Tabla de Contenido 2

Documentación del Proyecto 5

Resumen Ejecutivo . . . 5

Stack Tecnológico . . . 5

Estadísticas del Proyecto . . . 5

2. Documentación de Arquitectura y Diseño . . . 6

2.1 Estructura de Directorios . . . 6

2.2 Distribución por Capas . . . 6

2.3 Distribución de Lenguajes . . . 6

2.4 Diagrama de Infraestructura . . . 6

2.5 Diagrama de Casos de Uso . . . 6

2.6 Diagrama de Clases . . . 7

2.7 Patrones Detectados . . . 7

2.8 Insights de Arquitectura (IA) . . . 7

3. Documentación Técnica de Código . . . 7

3.1 Guía de Configuración del Entorno de Desarrollo . . . 7

3.2 Documentación de API . . . 8

3.3 Estado de Comentarios y Docstrings . . . 10

3.4 Planes de Prueba (Testing) . . . 10

4. Documentación de Clases . . . 11

`UserModel` . . . 11

`ProjectModel` . . . 11

`ReportSchema` *12*

5. Documentación de Funciones 12

Módulo: `routes/auth.py` *12*

Módulo: `services/exporter.py` *13*

6. Modelos de Datos 13

`UserModel` *13*

`ProjectModel` *13*

`ReportSchema` *13*

7. Guía de Despliegue y Mantenimiento 14

7.1 Requisitos del Sistema *14*

7.2 Instalación Local *14*

7.3 Despliegue con Docker *14*

7.4 Variables de Entorno Recomendadas *14*

7.5 Checklist de Despliegue *15*

7.6 Registro de Cambios (Changelog) *15*

8. Reporte de Calidad del Código 15

8.1 Análisis de Métricas *15*

8.2 Funciones con Alta Complejidad Ciclomática *16*

8.3 Recomendaciones *16*

8.4 Errores de Análisis *16*

9. Análisis de Datos del Proyecto 16

9.1 Tipos y Fuentes de Datos Detectados *16*

9.2 Esquema de Data Warehouse Inferido *16*

9.3 Reporte de Calidad de Datos *17*

9.4 Minería de Datos Aplicada al Código *17*

9.5 Pipeline ETL Utilizado 18

AutoDocs Sistema de Gestión y APIs

Documentación Técnica · AutoDocs AI · 16/10/2026

18	2	3	4
Archivos	Funciones	Clases	Endpoints

Score de Calidad: 88/100

Documentación del Proyecto

Generado automáticamente por AutoDocs AI — 2026-10-16 03:01 UTC

Resumen Ejecutivo

Este proyecto está desarrollado principalmente en **Python** y cuenta con 18 archivos fuente, 2 funciones, 3 clases y 4 endpoints de API documentados. El score de calidad del código es **88/100**.

Stack Tecnológico

Lenguaje	Cantidad

Estadísticas del Proyecto

Métrica	Valor
Total de archivos	18
Funciones	2
Clases	3
Endpoints API	4
Score de calidad	88/100
Funciones documentadas	1 / 2
Clases documentadas	2 / 3

2. Documentación de Arquitectura y Diseño

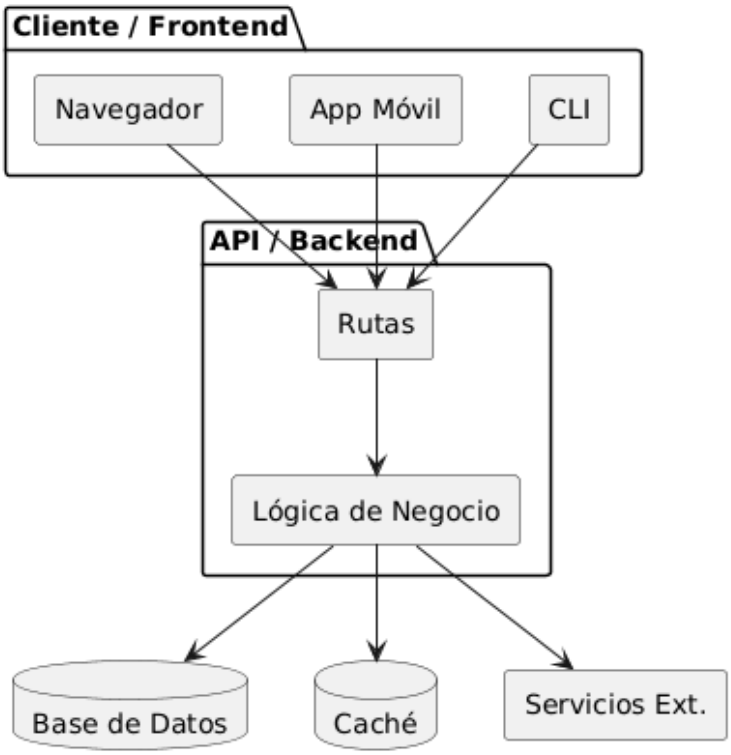
2.1 Estructura de Directorios

```
proyecto/
```

2.2 Distribución por Capas

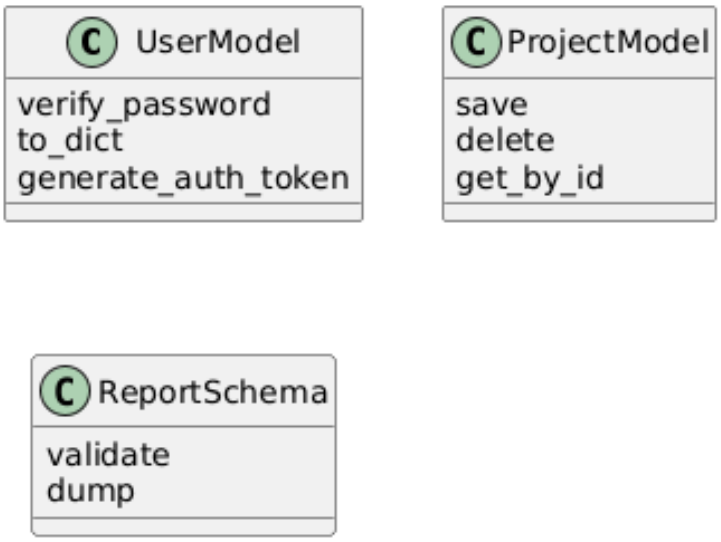
2.3 Distribución de Lenguajes

2.4 Diagrama de Infraestructura



2.5 Diagrama de Casos de Uso

2.6 Diagrama de Clases



2.7 Patrones Detectados

• **API REST**: 4 endpoints detectados (Flask)

• **Patrón MVC**: clases Model detectadas

2.8 Insights de Arquitectura (IA)

No se generaron insights de IA - configura GROQ_API_KEY en el archivo .env para habilitar esta función.

---

3. Documentación Técnica de Código

3.1 Guía de Configuración del Entorno de Desarrollo

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Crear entorno virtual (recomendado para Python)
python -m venv venv
source venv/bin/activate  # Linux/Mac
venv\Scripts\activate     # Windows

# 3. Instalar dependencias
pip install -r requirements.txt
```

```
# 4. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores locales

# 5. Ejecutar en modo desarrollo
python app.py  # o npm run dev
```

3.2 Documentación de API

Total de endpoints detectados: **4**

Módulo: `routes/auth.py`

[POST] `POST /api/v1/auth/login`

• **Framework**: Flask

• **Archivo**: `routes/auth.py`

Parámetros esperados:

• `body` (JSON, requerido): Cuerpo de la solicitud

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

Módulo: `routes/api.py`

[GET] `GET /api/v1/projects`

• **Framework**: Flask

• **Archivo**: `routes/api.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

- `200` — Operación exitosa
- `400` — Solicitud incorrecta
- `401` — No autorizado
- `404` — Recurso no encontrado
- `500` — Error interno del servidor

[POST] `POST /api/v1/projects/generate-docs`

- **Framework**: Flask
- **Archivo**: `routes/api.py`

Parámetros esperados:

- `body` (JSON, requerido): Cuerpo de la solicitud
- `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

- `200` — Operación exitosa
- `400` — Solicitud incorrecta
- `401` — No autorizado
- `404` — Recurso no encontrado
- `500` — Error interno del servidor

[GET] `GET /api/v1/projects/{id}/export/pdf`

- **Framework**: Flask
- **Archivo**: `routes/api.py`

Parámetros esperados:

- `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

- `200` — Operación exitosa
- `400` — Solicitud incorrecta
- `401` — No autorizado
- `404` — Recurso no encontrado

• `500` — Error interno del servidor

3.3 Estado de Comentarios y Docstrings

Total de funciones: 2

• **[OK] Documentadas:** 1 (50%)

• **[X] Sin documentar:** 1 (50%)

Visualización de documentación:

```
[■■■■■■■■■■■■■■■■■■■■■■]
50% documentado
```

Ejemplo de docstrings detectados:

```
def authenticate_user(username, password):
    """
    Valida credenciales contra la base de datos.
    """
    ...
```

Funciones que requieren documentación:

• `build_pdf_report(doc_data, output_path)` en `services/exporter.py`

3.4 Planes de Prueba (Testing)

No se detectaron pruebas automatizadas. Se recomienda implementar:

```
# Ejemplo de estructura de tests recomendada
import pytest

class TestEndpoints:
    def test_get_returns_200(self, client):
        response = client.get('/api/endpoint')
        assert response.status_code == 200

    def test_post_creates_resource(self, client):
        data = {'name': 'test'}
        response = client.post('/api/endpoint', json=data)
        assert response.status_code == 201

    def test_unauthorized_returns_401(self, client):
        response = client.get('/api/protected')
        assert response.status_code == 401
```

Escenarios de prueba recomendados:

Escenario	Tipo	Prioridad
POST /api/v1/auth/login — respuesta 200	Integración	Alta
POST /api/v1/auth/login — sin auth (401)	Seguridad	Alta
GET /api/v1/projects — respuesta 200	Integración	Alta
GET /api/v1/projects — sin auth (401)	Seguridad	Alta
POST /api/v1/projects/generate-docs — respuesta 200	Integración	Alta
POST /api/v1/projects/generate-docs — sin auth (401)	Seguridad	Alta
GET /api/v1/projects/{id}/export/pdf — respuesta 200	Integración	Alta
GET /api/v1/projects/{id}/export/pdf — sin auth (401)	Seguridad	Alta

---

4. Documentación de Clases

Total: **3 clases**

`UserModel`

• **Archivo:** `models/user.py`

• **Línea:** ?

• **Hereda de:** BaseModel

• **Métodos:** 3

• **Documentada:** [OK] Sí

Representa las credenciales y perfil de un usuario.

Métodos:

• `verify_password()`

• `to_dict()`

• `generate_auth_token()`

`ProjectModel`

• **Archivo:** `models/project.py`

• **Línea:** ?

• **Hereda de:** BaseModel

• **Métodos:** 3

• **Documentada:** [X] No

Métodos:

• `save()`

• `delete()`

• `get_by_id()`

`ReportSchema`

• **Archivo:** `models/report.py`

• **Línea:** ?

• **Hereda de:** Schema

• **Métodos:** 2

• **Documentada:** [OK] Sí

> **Esquema de serialización de reportes.**

Métodos:

• `validate()`

• `dump()`

---

5. Documentación de Funciones

Total: **2 funciones**

Módulo: `routes/auth.py`

`authenticate_user`

• Parámetros: `username, password`

• Async: —

• Complejidad: [LOW] 3

• Documentada: [OK]

> **Valida credenciales contra la base de datos.**

`authenticate_user(username, password)`

Ubicación: `routes/auth.py`

> **Valida credenciales contra la base de datos.**

Módulo: `services/exporter.py`

`build_pdf_report`

• Parámetros: `doc_data, output_path`

• Async: —

• Complejidad: [MED] 6

• Documentada: [X]

---

6. Modelos de Datos

Se detectaron **3 modelos de datos**:

`UserModel`

• Archivo: `models/user.py`

• Hereda de: BaseModel

• Métodos: verify_password, to_dict, generate_auth_token

`ProjectModel`

• Archivo: `models/project.py`

• Hereda de: BaseModel

• Métodos: save, delete, get_by_id

`ReportSchema`

• Archivo: `models/report.py`

• Hereda de: Schema

• Métodos: validate, dump

---

7. Guía de Despliegue y Mantenimiento

7.1 Requisitos del Sistema

• **Lenguaje principal**: Python

• **Archivos detectados**: 18

7.2 Instalación Local

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores

# 4. Ejecutar el proyecto
python app.py
```

7.3 Despliegue con Docker

```
FROM python:3.12-slim

WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8000
CMD ["python app.py"]
```

```
# Construir imagen
docker build -t mi-proyecto .

# Ejecutar contenedor
docker run -p 8000:8000 mi-proyecto
```

7.4 Variables de Entorno Recomendadas

```
# General
APP_ENV=production
SECRET_KEY=clave-secreta-segura
DEBUG=False

# Base de datos
DB_HOST=localhost
DB_PORT=5432
DB_NAME=mi_base_datos
DB_USER=usuario
DB_PASSWORD=contraseña

# API
API_VERSION=v1
ALLOWED_HOSTS=*
```

7.5 Checklist de Despliegue

• [ ] Variables de entorno configuradas

• [ ] Base de datos migrada

• [ ] Tests ejecutados y pasando

• [ ] Logs configurados

• [ ] HTTPS habilitado en producción

• [ ] Backups programados

• [ ] Monitoreo configurado

7.6 Registro de Cambios (Changelog)

Versión 1.0.0 — 2026-10-16

• Versión inicial analizada por AutoDocs AI

• Documentación técnica generada automáticamente

• Análisis de calidad del código completado

---

8. Reporte de Calidad del Código

8.1 Análisis de Métricas

Score de Calidad: 88/100 (Excelente [GREEN])

Distribución de Documentación:

• Funciones documentadas: 1/2 (50%)

• Clases documentadas: 2/3

Análisis de Complejidad:

• Complejidad promedio: 1

• Complejidad máxima: 0

• Funciones con alta complejidad (>10): 0

Tendencia de Complejidad por Archivo:

```
[Gráfico de regresión lineal mostrando la tendencia de complejidad]
Eje X: Archivos ordenados por complejidad promedio
Eje Y: Score de complejidad
Tendencia: ↓ Baja
```

8.2 Funciones con Alta Complejidad Ciclomática

No hay funciones con complejidad > 10. [OK]

8.3 Recomendaciones

• El código tiene buena calidad general. ¡Mantén las buenas prácticas!

8.4 Errores de Análisis

Sin errores detectados. [OK]

---

9. Análisis de Datos del Proyecto

9.1 Tipos y Fuentes de Datos Detectados

Datos Estructurados: 0 archivos

Datos Semi-estructurados: 0 archivos

Datos No Estructurados: 0 archivos

9.2 Esquema de Data Warehouse Inferido

No se detectaron suficientes modelos de datos para generar un esquema de Data Warehouse.

9.3 Reporte de Calidad de Datos

Problemas Detectados en el Código:

• **Funciones sin documentar:** 1

- `build_pdf_report` en `services/exporter.py`

9.4 Minería de Datos Aplicada al Código

Análisis automático realizado sobre las funciones y clases extraídas del proyecto.

A) Funciones de Alta Complejidad Ciclomática

Nivel	Umbral	Cantidad	Descripción
■ Alta	> 5	1	Requieren refactorización urgente
■ Media	4–5	0	Candidatas a simplificación
■ Baja	≤ 3	1	Complejidad aceptable

Funciones de alta complejidad — detalle y recomendaciones:

• `build_pdf_report [sin docstring]`

- ■ Archivo: `services/exporter.py` — línea ?

- ■ Complejidad ciclomática: **6**

- ■ Sugerencia: Complejidad moderada-alta (6). Revisa si alguno de los bloques if/for puede extraerse a una función con nombre descriptivo.

B) Funciones Sin Documentación

1 de 2 funciones carecen de docstring (50%).

Función	Archivo	Línea	Complejidad
`build_pdf_report`	`services/exporter.py`	?	6

■ **Sugerencia:** Agrega docstrings a todas las funciones públicas. Prioriza las de mayor complejidad primero, ya que son las más difíciles de entender sin documentación.

C) Detección de Funciones con Nombres Similares (Posibles Duplicados)

Se comparan los nombres de funciones entre archivos para detectar lógica duplicada o que podría unificarse.

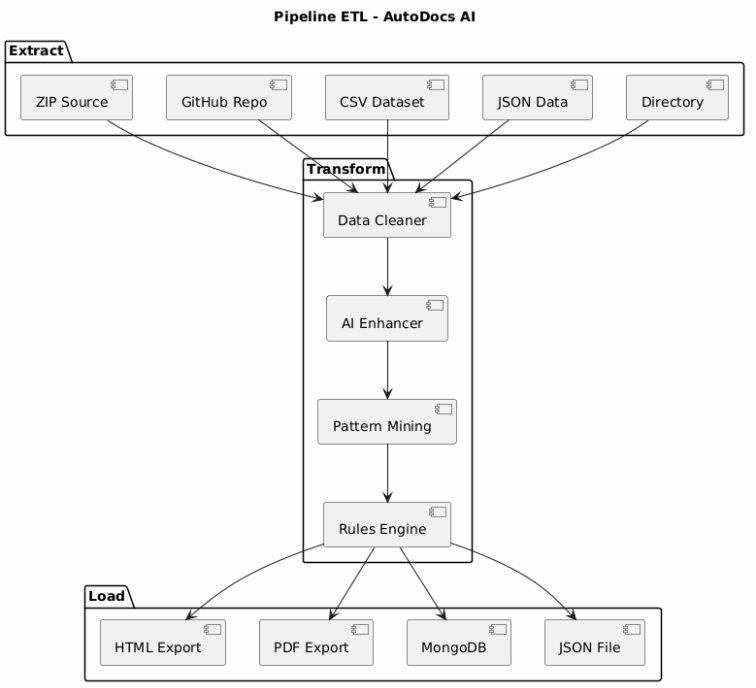
■ **No se detectaron funciones con nombres similares en archivos distintos.**

D) Análisis de Módulos — Archivos con Mayor Concentración de Funciones

Archivo	Funciones	Complejidad Promedio	Documentadas	Estado
`routes/auth.py`	1	3.0	100%	■ OK
`services/exporter.py`	1	6.0	0%	■ Revisar

9.5 Pipeline ETL Utilizado

El proceso de generación de documentación sigue un pipeline ETL (Extract, Transform, Load):



Fases del Pipeline:

• **Extract:** Extracción de código desde repositorios GitHub, archivos ZIP o directorios locales

• **Transform:** Análisis del código, limpieza de datos, enriquecimiento con IA y minería de patrones

• **Load:** Generación de documentación en HTML, PDF o almacenamiento en MongoDB

Documentación generada automáticamente por AutoDocs AI — 2026-10-16 03:01 UTC